Operating Systems and Parallel Computing

Lecture 10

Prof. Matteo Turilli

Users, Groups and File Ownership

- The file system stores data and metadata, while the OS checks which processes may access them.
- A user account has a numeric **user ID (UID)**. Processes run with user identities, which the OS uses to check access.
- A group has a **group ID (GID)** and can include several users. Each user can belong to several groups.
- Each file and directory has one owner UID and one group GID. These are metadata associated with its inode.
- In our container, `id` reports UID `1001` and GID `1001` for `analyst`. The numbers match here, but identify different things.
- User and group names make these IDs easier to read. Permissions decide what those identities are allowed to do.

Reading Our Container's Directory Listing

```
analyst@ospc:~$ ls -la
total 32
drwxr-x---  1 analyst analyst 4096 Sep 29 19:36 .
drwxr-xr-x  1 root    root    4096 Aug 31 13:27 ..
-rw-r--r--  1 analyst analyst  220 Feb 13  2026 .bash_logout
-rw-r--r--  1 analyst analyst 3771 Feb 13  2026 .bashrc
-rw-r--r--  1 analyst analyst  807 Feb 13  2026 .profile
drwxr-xr-x  2 analyst analyst 4096 Aug 31 13:27 data
drwxr-xr-x  2 analyst analyst 4096 Aug 31 13:27 tools
drwx------+ 32 analyst analyst 1024 Sep 23 06:19 work
```

- `ls -la` gives a detailed listing, including hidden names. Remember: `.` means this directory and `..` means its parent.
- The two names after the link count are the owner and group. Use `ls -ln` to display their numeric UID and GID.

Nine Bits, Three Permission Classes

- Looking at the `.bashrc` file, `-rw-r--r--` starts with `-` for a regular file, followed by three triplets: `rw-`, `r--`, `r--`.
- The triplets apply to the **owner**, **group** and **others**, in that order. Others means users who match neither owner nor group.
- For a regular file, `r` allows reading, `w` allows changing its contents, and `x` allows execution. A dash means that permission is absent.
- The OS first checks the process's user identity. If it matches the file owner, only the owner triplet applies.
- Otherwise, a matching group uses the group triplet. If neither matches, the others triplet applies. The triplets are not added together.
- Here, `analyst` can read and edit `.bashrc`, while group members and others can read it, provided they can reach its directory.

Permissions on Directories

- On a directory, `r` lets a process list entry names, while `x` lets it look up those names and traverse the directory.
- Directory `w`, together with `x`, normally permits creating, renaming and removing entries. File `w` controls the contents instead.
- A file can therefore be deleted without write permission on that file, if its parent directory permits removal.
- The initial `d` means directory. Our home (`.`) is `drwxr-x---`, so the `others` class of users cannot reach files inside.
- The `+` on `work` signals extra access-control information, e.g., an access-control list (ACL). The nine bits may not describe all access rules.

Inspect Permissions Before Changing Them

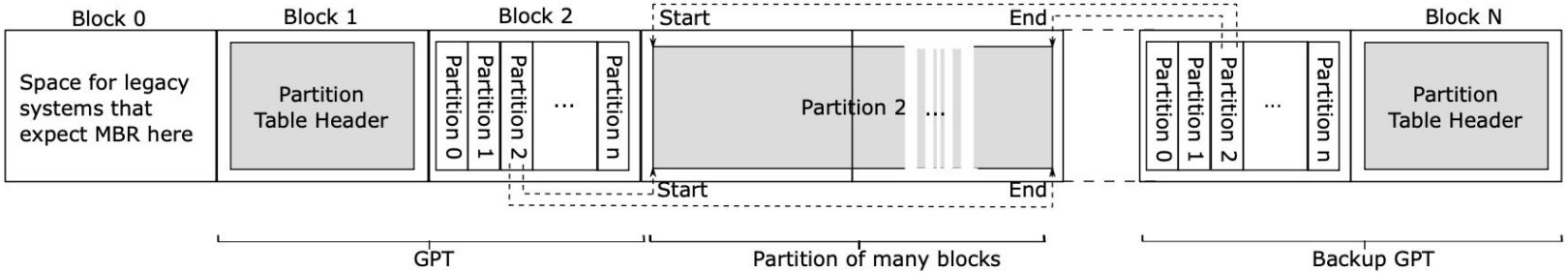
- Use `id` to inspect your user and groups, and `ls -l` to inspect a file's owner, group and permissions.
- Use `ls -ld data` to inspect the directory itself, rather than list its contents.
- An owner can use `chmod` to change permission bits. For example, on a practice file: `chmod u=rw,g=r,o= notes.txt`.
- This gives the owner read and write access, the group read access, and others no access. The result is `-rw-r-----`.
- In `chmod`, `u` means owner, `g` group and `o` others. `a` means all three classes, so others and all are different.
- Example: A loading of a csv file that fails with a permission error can reflect missing file read permission or directory search permission.

`sudo` Runs a Command as Another User

- `sudo` runs an authorized command as another user, usually `root` (UID `0`). The system's policy decides what is allowed.
- In our course container, `analyst` is allowed to use `sudo` without a password. Other systems may require a password or deny the command.
- Compare `id`, `sudo id` and then `id` again. The middle command runs as root, while your ordinary shell remains analyst.
- Use `sudo -l` to see what your account is permitted to run. It describes privileges, not a file's permission bits.
- Files created by a command run as root can belong to root, which may prevent later edits as analyst.
- The `work` folder is shared with your computer. Changes there affect shared files, even when the command runs inside the container.

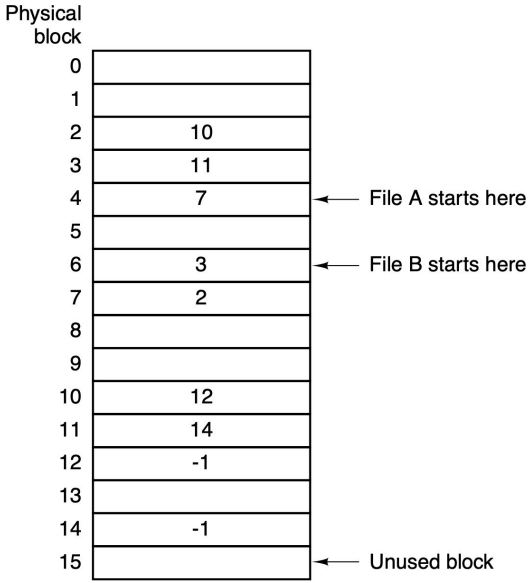
File System Implementation

- **Users** see names, paths, contents and access rules.
- **Implementers** organize data and metadata so that those operations work efficiently and reliably.
- An on-disk file system may occupy a partition, while other file systems use part of the RAM or remote storage shared over a network.
- The Globally Unique Identifier (GUID) Partition Table (GPT) below records partition locations. A filesystem describes the files inside each partition.
- During a typical computer boot, the BIOS loads a boot program and the OS later mounts file systems using its drivers.

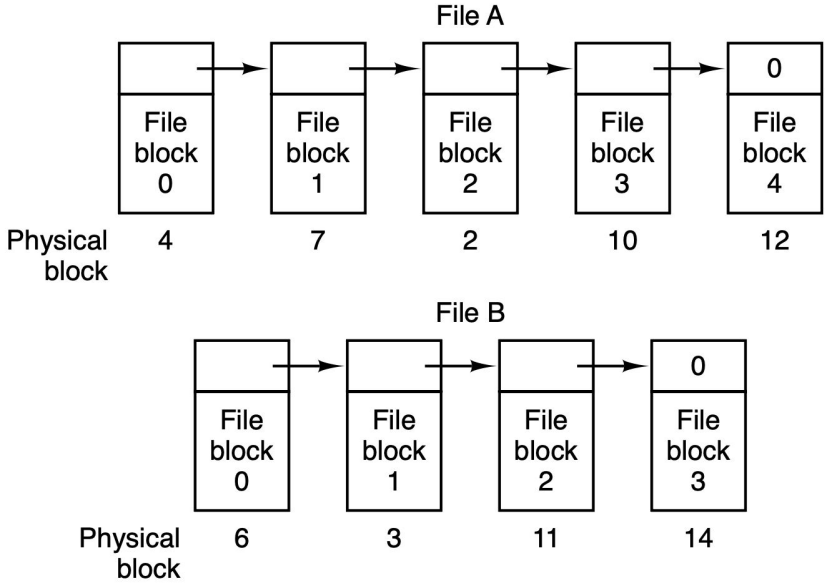


Implementing Files

- A file system must track which storage blocks belong to each file.
- A file allocation table (FAT) links files' allocation units, called clusters.
- The table is stored on disk and can be cached in RAM.



FAT cached in RAM



Files as chains of clusters

FAT: How a File Is Found

- Storage is divided into pieces of equal size, called clusters. The data of a file occupy one or more clusters.
- The clusters of a file do not have to be next to each other. FAT can use any free cluster, wherever it is, so a file can grow without needing one long empty region.
- The FAT has one entry for each cluster. The entry says which cluster comes next in the same file, or that this cluster is the last one (-1 in the figure).
- The directory entry of a file records two things: the number of its first cluster and the size of the file.
- To read a file, the OS goes to the first cluster, looks up its entry in the table to find the second, and repeats until it reaches the last. This sequence is the FAT chain.
- In the figure, File A starts at cluster 4. The table then leads to clusters 7, 2, 10 and 12, where the file ends.

FAT: Costs and Limitations

- Following a chain means reading many table entries. Reading them from disk would be slow, so the OS keeps a copy of the table in RAM, called a cache.
- The cache only speeds up finding where the data are. The data themselves must still be read from storage.
- The table has one entry per cluster, so a larger disk means a larger table and more RAM for the cache.
- Example: A 1 TB disk with 4 KB clusters has about 250 million clusters. At 4 bytes per entry, the table takes about 1 GB, half the RAM of our container.
- When RAM is short, the OS keeps in RAM only the parts of the table in use and reads the rest from disk when needed.
- The last cluster of a file is rarely full, so some space in it stays unused.
- The clusters of a file can be far apart, which slows reading on a hard disk as seeking the start of the following cluster costs time (especially a mechanical one).

Alternatives to FAT

- Other file systems avoid the chain used by FAT.
- Some keep, for each file, a list of all its blocks. Others record multiple extents for each file: "start at this block and continue for this many blocks".
- Example: ext4 is a common Linux file system that stores the starting point of a file's extent list in its inode.
- Journaling protects the file system's own records:
  a. It writes a note of what it is about to do before changing a record.
  b. It can finish or undo a half-made change after a crash.
- Note: Journaling is not a backup. Data written just before a crash can still be lost.

FAT chain: 5 entries
`4 → 5 → 6 → 7 → 8`

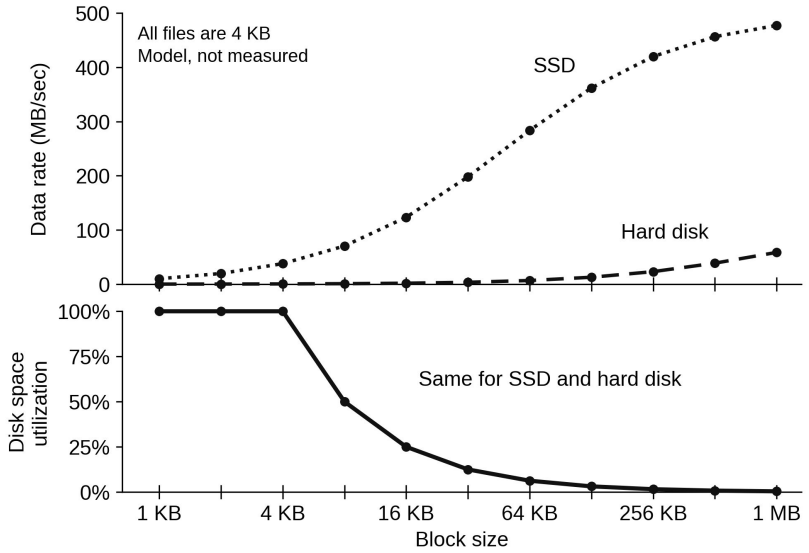
Per-file index: 1 list
`4, 5, 6, 7, 8`

Extent: 1 record
`start 4, length 5`

The same file, stored in blocks 4 to 8, described in three ways

File System Management and Optimization

- File-system design balances efficiency, space and reliability.
- How large should each allocation block be?
  - **Top panel: reading speed**. Each block costs one access to the device, so larger blocks bring more data per access.
  - **Bottom panel: the share of allocated space that holds data**. A 4KB file fills a 4 KB block, 1/2 of an 8 KB and 1/4 of a 16 KB.
  - Choose the block size for the workload: small blocks for many small files, larger blocks for few large files.



- Other decisions:
  - Tracking free space and allocating data.
  - Quotas to control how much storage each user or group can consume.
  - Backups to recover data after deletion, corruption or device failure.
  - Consistency checks and repairs, which can involve data loss.
  - Caching to reduce repeated storage access.
  - Compression and deduplication to reduce the space required.

This is an interactive class

- Seven exercises, all typed in the container's shell. No Python is needed.
- Start a shell in the course container, then create a scratch folder and move into it:

```
docker compose run --rm ospc
mkdir ~/lab
cd ~/lab
```

- We work in `~/lab` and not in `work`. The `work` folder is shared with your computer, whose file system may treat links and permissions differently from Linux.
- Each exercise has three steps:
  a. Make your prediction,
  b. Run the commands,
  c. Answer the questions.
- Type the commands one at a time.
- Several commands fail on purpose: the error message is part of the exercise.

Exercise 1: Bytes, Characters, Extensions

- The OS stores bytes. Programs decide what the bytes mean.
- **Predict**: The word `Málaga` has six letters. With a newline at the end, how many bytes will the file contain?

```
printf 'Málaga\n' > city.txt
wc -c city.txt
od -Ax -t x1z city.txt
```

- **Predict**: Does changing the extension of a file change its content?

```
cp city.txt city.pdf
cat city.pdf
head -c 4 city.pdf | od -c
head -c 4 city.txt | od -c
```

Exercise 1: Questions

1. Which letter of needs more than one byte?
2. Which bytes are they?
3. Is city.pdf a PDF?
4. What did cp change, and what did it leave unchanged?

Exercise 1: Answers

1. Which letter of needs more than one byte?
2. Which bytes are they?
3. Is city.pdf a PDF?
4. What did cp change, and what did it leave unchanged?

1. á.
2. á is stored as `c3 a1` in UTF-8. Every other letter is one ASCII byte.
3. No.
4. `cp` copied the same 8 bytes under a new name. `cat` still prints text, and the first bytes are `M á l`, not the `%PDF` signature of a PDF.

The Curious Corner

- Are you curious to see how a pdf file looks like?

```
sudo apt update; sudo apt install -y enscript ghostscript
iconv -f UTF-8 -t ISO-8859-1 city.txt | enscript -B -p city.ps
ps2pdf city.ps city_real.pdf
head -c 4 city_real.pdf | od -c
```

- `iconv` encodes the file into `ISO-8859-1` because enscript does not understand the newer and more complete `UTF-8` encoding.
- `enscript` and `ps2pdf` write new files in other formats
- `cp` only gave the same bytes a new name.
- An encoding is the rule that maps characters to bytes.
- The format is how the bytes are organised inside the file.
- The extension is only part of the name.

Exercise 2: What the Inode Records

- A directory entry links a name to an inode number. The inode holds the metadata.
- Copy the dataset into the scratch folder and inspect the copy:

```
cp ~/data/sales_2025_sample.csv sales.csv
ls -li sales.csv
```

- In the `ls -li` output, which number is the inode and which is the link count?
- The next command prints inode number, size, permissions, modification time and change time. Press the up arrow to run it again after each command.

```
stat -c '%i  %s bytes  %A  modified %y  changed %z' sales.csv
```

- **Predict.** Which of the four fields change after each command?

```
echo 'extra row' >> sales.csv
chmod g+w sales.csv
mv sales.csv renamed.csv
```

Exercise 2: Questions

1. After the rename, which field shows that it is still the same file?
2. We did not edited the contents in the second and third actions. Why did the change time move?

Exercise 2: Answers

1. After the rename, which field shows that it is still the same file?
2. We did not edited the contents in the second and third actions. Why did the change time move?

1. The inode number. The name changed but the file did not.
2. Change time records updates to the inode, such as permissions or a rename. It is not a creation time.

Exercise 3: `cp`, `ln` and `ln -s`

- Three commands give a file a second name. Only one makes a second file.

```
echo 'version 1' > draft.txt
cp draft.txt copy.txt
ln draft.txt hard.txt
ln -s draft.txt soft.txt
ls -li draft.txt copy.txt hard.txt soft.txt
```

- **Predict.** We now add a line to `draft.txt` and change its permissions. Which of the file, hard and soft link show `version 2`? Which show the new permissions?

```
echo 'version 2' >> draft.txt
cat copy.txt
cat hard.txt
cat soft.txt
chmod 600 draft.txt
ls -li draft.txt copy.txt hard.txt soft.txt
```

Exercise 3: Questions

1. Which type of files share an inode number?
2. What is the link count of `draft.txt`, and why?
3. `soft.txt` is 9 bytes long. Which 9 bytes does it store?
4. `chmod` was applied to `draft.txt` only. Why did `hard.txt` change as well?
5. Between `draft.txt` and `hard.txt`, which one is the original?

Exercise 3: Answers

1. Which type of files share an inode number?
2. What is the link count of `draft.txt`, and why?
3. `soft.txt` is 9 bytes long. Which 9 bytes does it store?
4. `chmod` was applied to `draft.txt` only. Why did `hard.txt` change as well?
5. Between `draft.txt` and `hard.txt`, which one is the original?

1. A normal file and a hard link, e.g., `draft.txt` and `hard.txt`.
2. The link count is 2 because two directory entries point to that inode.
3. None, it stores the target path, i.e., `draft.txt`. A symbolic link is a small file whose content is a path.
4. Permissions are stored in the inode, and both names point to the same inode.
5. Neither. The two names are equal: there is one file with two names.

Exercise 4: Removing a File Name

- `rm` removes a file name. Whether the file disappears depends on how many names that file has, e.g., hard/soft links.
- **Predict.** After `rm draft.txt`, can we still read `hard.txt`? And `soft.txt`?

```
rm draft.txt
ls -li
cat hard.txt
cat soft.txt
```

- **Predict.** We create a new `draft.txt` with different contents. What will `hard.txt` and `soft.txt` show?

```
echo 'brand new' > draft.txt
ls -li draft.txt hard.txt
cat hard.txt
cat soft.txt
```

Exercise 4: The Cost of a Second Name

- Last step: does a second name cost storage?
- `renamed.csv` is the 6 MB file from Exercise 2, and `du -sh .` reports the storage used by the folder.
- **Predict.** By how much will the folder grow after `ln`, and after `cp`?

```
du -sh .
ln renamed.csv sales_hard.csv
du -sh .
cp renamed.csv sales_copy.csv
du -sh .
```

Exercise 4: Questions

1. What happened to the link count of `hard.txt` when `draft.txt` was removed?
2. When would the data be reclaimed?
3. Is the new `draft.txt` the same file as `hard.txt`? Which number tells us?
4. Why did `soft.txt` fail after the `rm` and work again after the `echo`?

Exercise 4: Answers

1. What happened to the link count of `hard.txt` when `draft.txt` was removed?
2. When would the data be reclaimed?
3. Is the new `draft.txt` the same file as `hard.txt`? Which number tells us?
4. Why did `soft.txt` fail after the `rm` and work again after the `echo`?

1. It dropped from 2 to 1. `rm` calls `unlink()`, which removes one name.
2. Data are reclaimed when the last hard link and the last open reference are gone.
3. No. The new `draft.txt` has a different inode number.
4. `soft.txt` stores a path. After `rm` the path did not resolve, so the link was dangling. The new file made the path resolve again, to a different file.

Exercise 5: Absolute and Relative Paths

- A relative path starts from the current working directory. An absolute path starts from the root.
- **Predict.** You are in `/home/analyst/lab`. Which of these four paths will find the dataset?

```
pwd
wc -c data/sales_2025_sample.csv
wc -c ../data/sales_2025_sample.csv
wc -c /home/analyst/data/sales_2025_sample.csv
wc -c ~/data/sales_2025_sample.csv
```

Exercise 5: The Special Names `.` and `..`

- Now change directory and look at the special names `.` and `..`:

```
cd ../data
pwd
wc -c sales_2025_sample.csv
cd ~/lab
ls -id . ~/lab ../lab
cd /
cd ..
pwd
ls -id / /..
cd ~/lab
```

Exercise 5: Questions

1. Why did the first `wc` fail from `~/lab` although the same file name worked after `cd ../data`?
2. `.`, `~/lab` and `../lab` print the same inode number. What does that tell you?
3. Where does `cd ..` take you from `/`? How do the inode numbers confirm it?
4. A script loads `data/sales.csv`. What must be true when it starts? How would you make it work from any directory?

Exercise 5: Answers

1. Why did the first `wc` fail from `~/lab` although the same file name worked after `cd ../data`?
2. `.`, `~/lab` and `../lab` print the same inode number. What does that tell you?
3. Where does `cd ..` take you from `/`? How do the inode numbers confirm it?
4. A script loads `data/sales.csv`. What must be true when it starts? How would you make it work from any directory?

1. A relative path is resolved from the current working directory. From `~/lab` the OS looked for `~/lab/data/...`
2. They are three paths to the same directory. A file or directory can be reached by many paths.
3. Nowhere: /.. refers to / itself (slide 22). Both print the same inode number.
4. The script must be started from the directory that contains data. An absolute path works from any directory.

Exercise 6: Permission Bits and `chmod`

- Every file has one owner, one group and nine permission bits. Start by reading who you are and what your home directory contains:

```
id
ls -la ~
ls -ln ~
```

- **Predict.** What are the ten-character permission string that `ls -l` shows after each `chmod`.

```
echo 'my notes' > notes.txt
chmod u=rw,g=r,o= notes.txt
ls -l notes.txt
chmod a+x notes.txt
ls -l notes.txt
chmod o-x notes.txt
ls -l notes.txt
chmod a=r notes.txt
ls -l notes.txt
```

Exercise 6: Owner, Group or Others?

- The OS uses exactly one of the three triplets for each access.
- **Predict.** The next file gives read permission to the group and to others, but not to the owner. You own it and you belong to its group. Can you read it?

```
echo 'Q4 revenue: confidential' > report.txt
chmod u=,g=r,o=r report.txt
ls -l report.txt
cat report.txt
chmod u+r report.txt
cat report.txt
```

Exercise 6: Questions

1. What are your UID and GID in our container?
2. What is the difference between `o` and `a` in `chmod`?
3. The group triplet of `report.txt` was `r--` and you are in that group. Why were you refused?
4. You could not read `report.txt`, yet `chmod u+r` worked. What gives you the right to change its permissions?

Exercise 6: Answers

1. What are your UID and GID in our container?
2. What is the difference between `o` and `a` in `chmod`?
3. The group triplet of `report.txt` was `r--` and you are in that group. Why were you refused?
4. You could not read `report.txt`, yet `chmod u+r` worked. What gives you the right to change its permissions?

1. UID 1001 and GID 1001. `ls -l` shows names; `ls -n` shows the numbers.
2. `o` is the others class only. `a` is all three classes: owner, group and others.
3. The OS checks owner, then group, then others, and stops at the first match. You are the owner, so only the owner triplet --- applies.
4. Ownership. Changing permission bits requires being the owner, not having read or write permission.

Exercise 7: Permissions on Directories

- On a directory, `r` lists the names, `x` looks a name up, and `w` with `x` creates and removes entries.
- Build a directory with two small files. We will change only the directory's permissions. You own `vault`, so only its owner triplet applies to you.

```
mkdir vault
echo 'a,b' > vault/a.csv
echo 'c,d' > vault/b.csv
ls -ld vault
```

- **Predict.** For each owner triplet on `vault` (`r--`, then `--x`, then `r-x`): can you list the names, read `a.csv`, append to `a.csv`, remove `a.csv`?

Exercise 7: Three Triplets, Four Tests

- Run the same four tests after each `chmod`:

```
chmod u=r vault
ls vault
cat vault/a.csv
echo 'e,f' >> vault/a.csv
rm vault/a.csv
```

- Repeat them after `chmod u=x vault`, then after `chmod u=rx vault`.
- **Predict.** We restore `rwx` on `vault` and make `b.csv` read-only. Can we append to it? Can we delete it?

```
chmod u=rwx vault
chmod a-w vault/b.csv
echo 'x,y' >> vault/b.csv
rm -f vault/b.csv
ls -l vault
```

Exercise 7: Questions

1. In Step 2 `ls vault` failed, but `cat vault/a.csv` worked. What does that tell you about `x` on a directory?
2. In Step 3 you could change the contents of `a.csv` but not remove it. Which permission controls each operation?
3. Why could you delete `b.csv` although nobody had write permission on it?
4. A CSV fails to load with "Permission denied". Which two things do you check, and with which commands?

Exercise 7: Answers

1. In Step 2 `ls vault` failed, but `cat vault/a.csv` worked. What does that tell us about `x` on a directory?
2. In Step 3 we could change the contents of `a.csv` but not remove it. Which permission controls each operation?
3. Why could you delete `b.csv` although nobody had write permission on it?
4. A CSV fails to load with "Permission denied". Which two things do you check, and with which commands?

1. `x` lets the OS follow a name to its inode. Without `r` the names cannot be listed, but a file whose name we already know can still be opened.
2. The file's `w` controls its contents. The directory's `w`, together with `x`, controls creating and removing entries.
3. Deleting changes the directory, not the file. The directory had `w` and `x`, so the entry could be removed.
4. Read permission on the file with `ls -l file`, and search permission on each directory of its path with `ls -ld directory`.